## Ejercicios que desarrollamos en la clase del jueves 3 de abril, 2025

**15. Mencione 3 motivos por los cuales un servidor de DNS no puede ser centralizado.**

Veamos qué dice Kurose en su página 136:

> how the DNS servers and querying hosts communicate.
>
> A simple design for DNS would have one DNS server that contains all the mappings. In this centralized design, clients simply direct all queries to the single DNS server, and the DNS server responds directly to the querying clients. Although the simplicity of this design is attractive, it is inappropriate for today's Internet, with its vast (and growing) number of hosts. The problems with a centralized design include:
>
> - **A single point of failure.** If the DNS server crashes, so does the entire Internet!
> - **Traffic volume.** A single DNS server would have to handle all DNS queries (for all the HTTP requests and e-mail messages generated from hundreds of millions of hosts).
> - **Distant centralized database.** A single DNS server cannot be "close to" all the querying clients. If we put the single DNS server in New York City, then all queries from Australia must travel to the other side of the globe, perhaps over slow and congested links. This can lead to significant delays.
> - **Maintenance.** The single DNS server would have to keep records for all Internet hosts. Not only would this centralized database be huge, but it would have to be updated frequently to account for every new host.

Es decir, la escala del problema sugiere pensar en un diseño descentralizado que permita mayor tolerancia a fallas parciales y mayor performance a expensas del modelo de consistencia para la base de datos del DNS (en este caso, distribuida).

Según la RFC 1034 (año ~ 1987), la escala del problema a resolver hacía impráctico o directamente imposible tener toda la base de datos de nombres globales en un único lugar con la tecnología de la época:

> *The sheer size of the database and frequency of updates suggest that it must be maintained in a distributed manner, with local caching to improve performance. Approaches that attempt to collect a consistent copy of the entire database will become more and more expensive and difficult, and hence should be avoided. The same principle holds for the structure of the name space, and in particular mechanisms for creating and deleting names; these should also be distributed.*

Previo a que exista DNS, existía una base de datos centralizada (`HOSTS.TXT`) mantenida por el Network Information Center (NIC) de la época. Ese archivo se distribuía periódicamente, vía protocolo FTP (*file transfer protocol*, esencialmente un protocolo que

hoy es considerado obsoleto para acceder remotamente a un sistema de archivos y poder bajar/subir archivos), a todos los hosts que conformaban la red en ese momento.

> *The total network bandwidth consumed in distributing a new version by this scheme is proportional to the square of the number of hosts in the network, and even when multiple levels of FTP are used, the outgoing FTP load on the NIC host is considerable. Explosive growth in the number of hosts didn't bode well for the future.*

Mirando en retrospectiva el esquema de una única base centralizada hubiese presentado un problema de escalabilidad enorme. Por estas razones se descartó tempranamente en pro de DNS como una opción para resolver un problema inherentemente descentralizado como es el esquema de nombres en la internet global de hoy

**22. Enviar un mensaje de consulta DNS directamente desde el host en el que está trabajando para resolver la IP del servidor www.uba.ar mediante nslookup.**

Para resolver este ejercicio, vamos a desarrollar los pasos necesarios que haría el servidor de DNS local (*) para transformar el nombre **"www.uba.ar"** en uno o más direcciones IPs.

(*) Aquí *local* se refiere a como lo define Kurose en el capítulo 2, es decir, al servidor de *caching* local a la estación de trabajo (*requesting host*) que típicamente forma parte de un setup tradicional en una oficina, hotel, hogar etc.

Para esto, vamos a usar como referencia el siguiente diagrama que modificamos usando como base la figura 2.19 de la sección 2.4 (DNS) del libro:

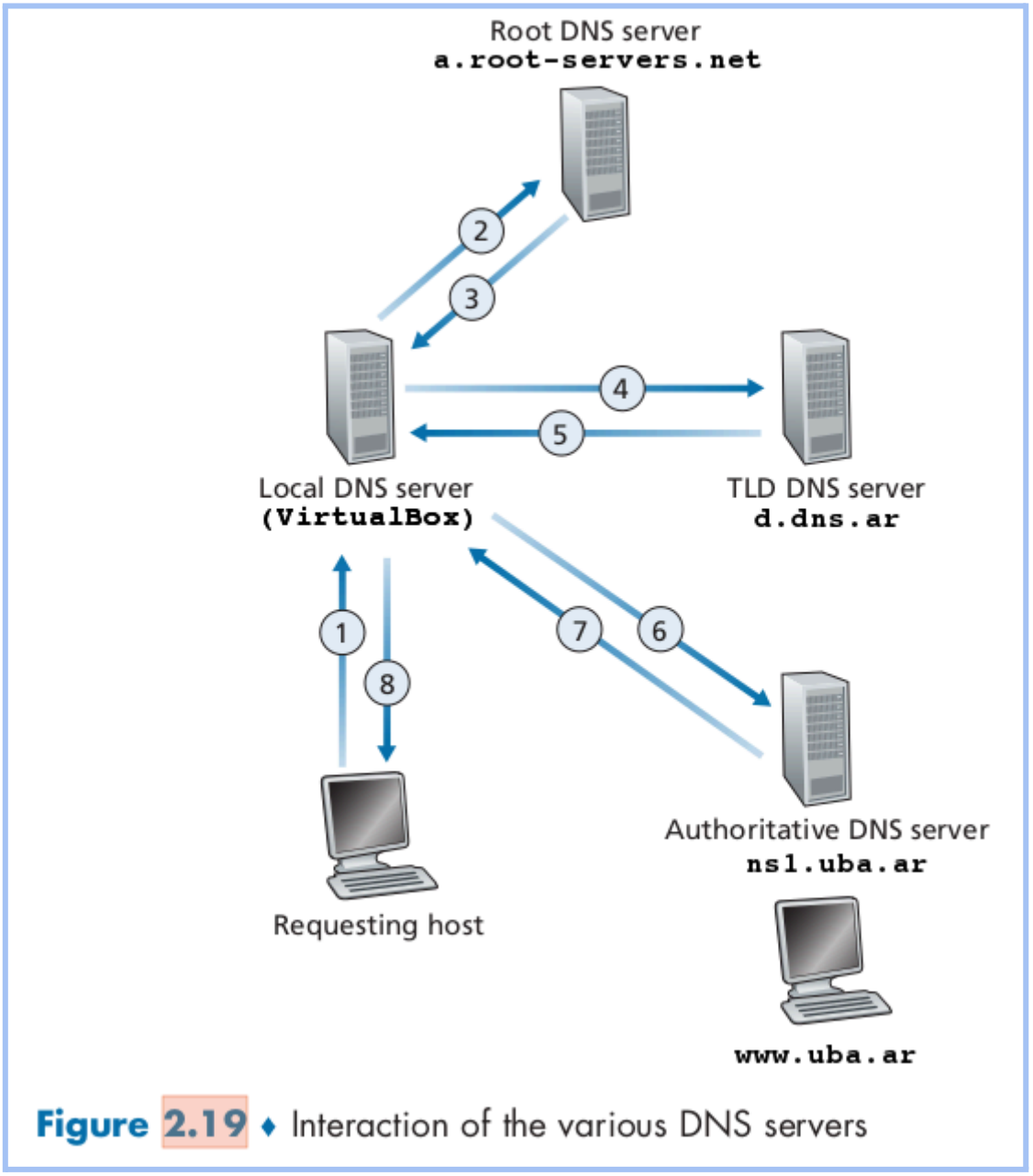



**Figura 1: arquitectura conceptual para resolver el ejercicio 22**

**Nota**: en el desarrollo que sigue (**pasos 1 a 7**), vamos a hacer de cuenta que somos el servidor local de DNS marcado como **`VirtualBox`** en la figura de arriba. Es decir, si bien los comandos dig de más abajo los vamos a ejecutar físicamente en el **Requesting host** del diagrama, las consultas de DNS generadas por dig imitarán lo que haría un Local DNS server en la misma situación. Más abajo (en el **paso 8**), vamos a repetir el experimento enviando una consulta al verdadero Local DNS server que provee VirtualBox.

**Importante**: se asume que estás usando la imagen de VirtualBox tal como es provista en el campus (i.e. archivo ComDatos_2022-08-25.ova). Para poder resolver este ejercicio en la VM de VirtualBox es necesario ejecutar los siguientes comandos:

```
apt update -y
apt install dnsutils -y
apt install dns-root-data -y
```

👆 El primer comando se usa para descargar la configuración de paquetes de las fuentes configuradas en Debian. El segundo comando es necesario para instalar la herramienta dig (no viene instalada por defecto en la imagen del campus). El tercer comando nos provee del archivo `root.hints` que vamos a usar más abajo para resolver las consultas de DNS que nos piden.

**Paso 1**. Asumiendo que somos **VirtualBox**, vamos a hacer de cuenta que recibimos una consulta de DNS en la cual el **Requesting host** nos pregunta por la dirección IP del sitio web de la UBA: "**www.uba.ar**".

Para transformar el nombre "**www.uba.ar**" en una dirección IP, vamos a ejecutar los siguientes comandos en la terminal del Ubuntu para simular el procedimiento de resolución de nombres que haría un software de DNS como el que está corriendo en el **Local DNS Server** de la figura.

Para esto, lo primero que hacemos es preguntarle a un servidor "root" de DNS por el servidor de DNS que tiene a cargo la zona "**ar**" en el DNS, es decir, buscamos la IP de los servidores de DNS que nos pueden dar respuestas autoritativas por consultas sobre el dominio "ar".

¿Cómo sabe el servidor de DNS local a quien dirigir esta primera consulta? Una manera de resolver esto en los programas que cumplen la función de caching DNS es configurando un archivo con una lista de servidores root a quienes consultar. En el caso del ambiente de VirtualBox que estamos usando, podemos inspeccionar el archivo "**`/usr/share/dns/root.hints`**" para obtener una lista preconfigurada de servidores de la zona "root" de DNS:

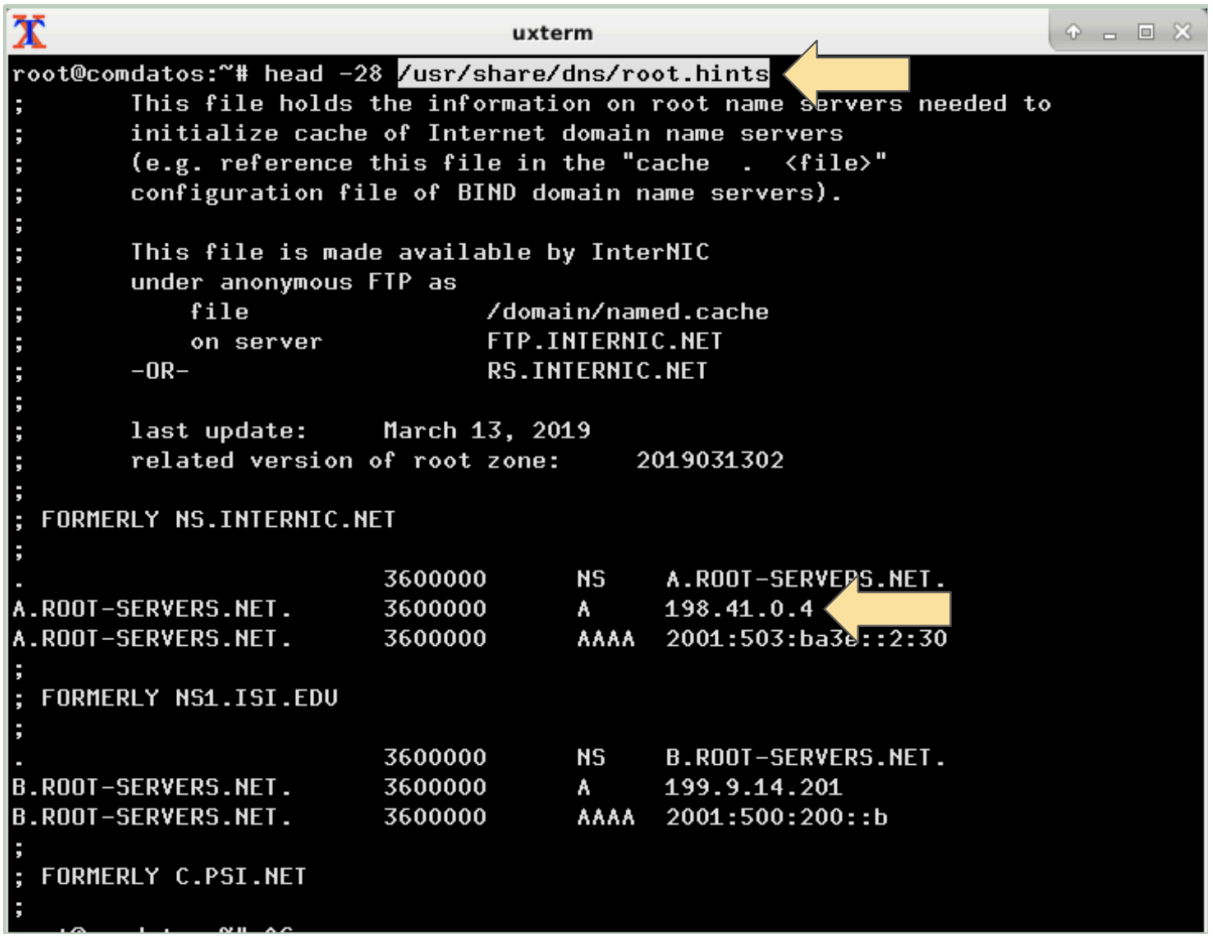



**Figura 2: encabezado de `/usr/share/dns/root.hints` (paso 1)**

**Paso 2**. Usando esta lista, vamos enviar una consulta a "**a.root-servers.net**" para averiguar cuáles son los servidores que tiene delegada la zona "**ar**" en el DNS, y que nos pueden dar respuestas autoritativas sobre cualquier nombre dentro de esa zona.

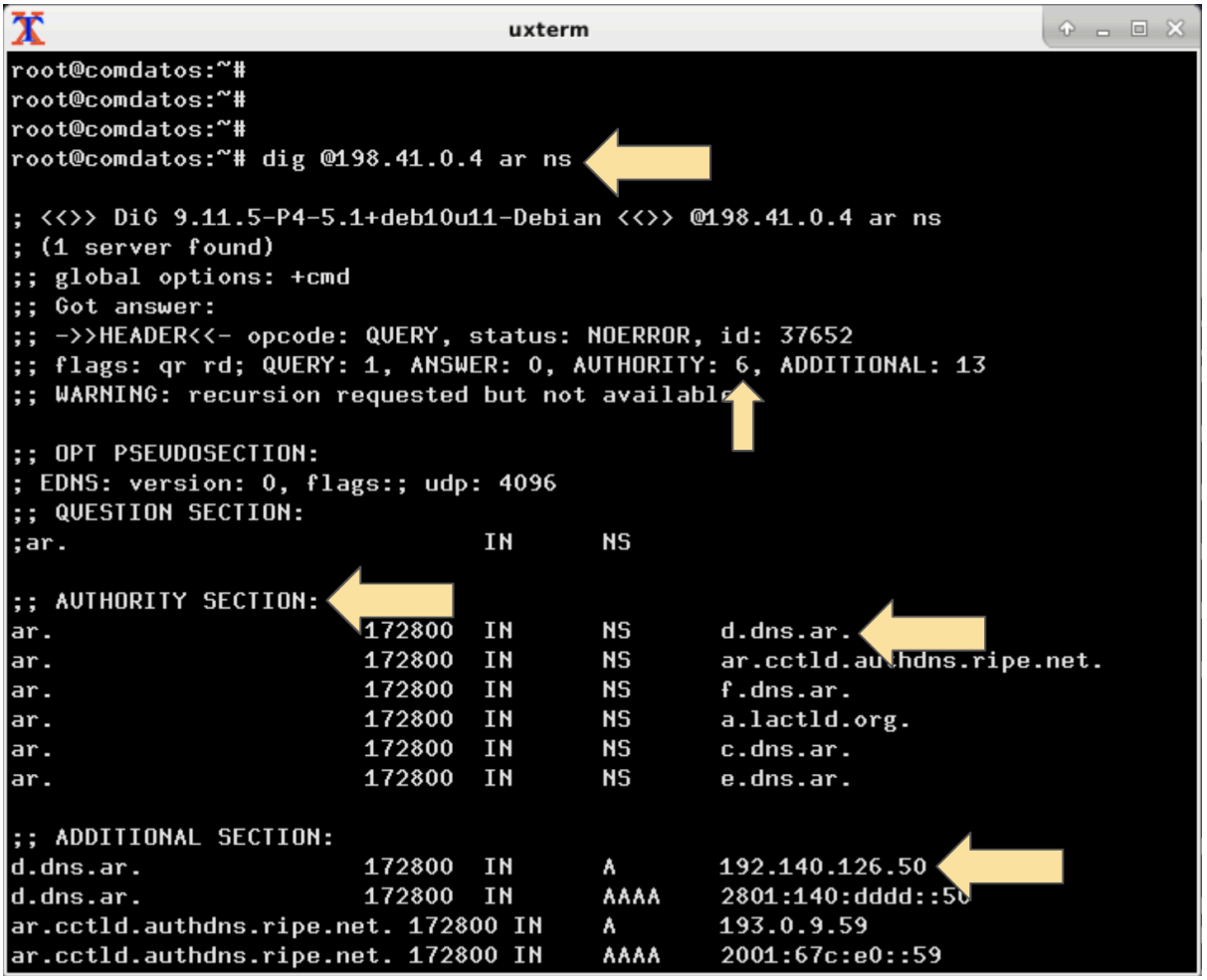



**Figura 3: consulta DNS y respuesta (pasos 2 y 3)**

**Paso 3.** En la respuesta que nos envía "**a.root-servers.net**" (que se corresponde con el paso 3 en el diagrama de la Figura 1 de arriba), podemos ver en la sección "authority" que el dominio "**ar**" se encuentra delegado a una serie de servidores de DNS allí enumerados. En particular, "**d.dns.ar**", para el cual además contamos con la dirección IP especificada en la sección "ADDITIONAL SECTION" de la respuesta (*).

(*) Si no estuviese esta IP, o si la respuesta al query del paso (3) no contase con una ADDITIONAL SECTION (afortunadamente no es el caso), entonces para obtener la dirección IP de "**d.dns.ar**" podríamos enviar una consulta adicional al servidor de DNS autoritativo para la zona "**dns.ar**" para preguntarle por el registro "A" de "**d.dns.ar**".

**Paso 4**. Ahora que tenemos la dirección IP de "**d.dns.ar**" (autoritativo para la zona "**ar**", es decir, nos puede dar respuestas por cualquier registro colgado de esa zona) lo que vamos a hacer es preguntarle por la delegación del dominio "**uba.ar**".

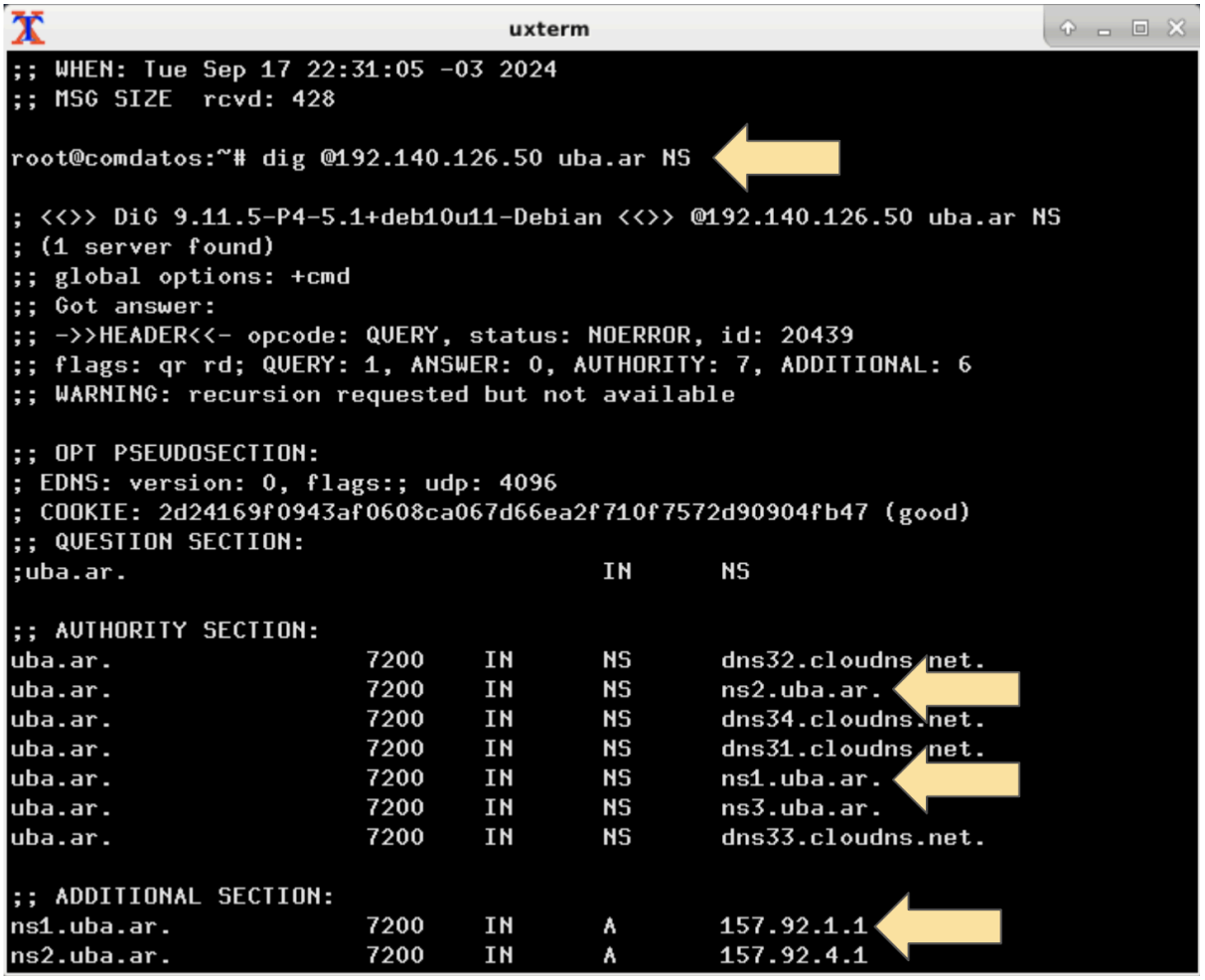



**Figura 4: consulta DNS y respuesta (pasos 4 y 5)**

**Paso 5**. En la respuesta que nos envía el servidor de DNS autoritativo para la zona "**ar**" podemos ver que "**ns1.uba.ar**" (157.92.1.1) y "**ns2.uba.ar**" (157.92.4.1) son algunos de los servidores de DNS que tienen delegada la zona "**uba.ar'.**

**Paso 6**. Vamos a elegir uno de estos nameservers (**ns1.uba.ar ~ 157.91.1.1**) para obtener la IP de "**www.uba.ar**". Típicamente, los servidores de DNS mezclan aleatoriamente los registros en sus respuestas, como así también seleccionan aleatoriamente a quién enviar sus consultas cuando tienen múltiples opciones, como es este caso.

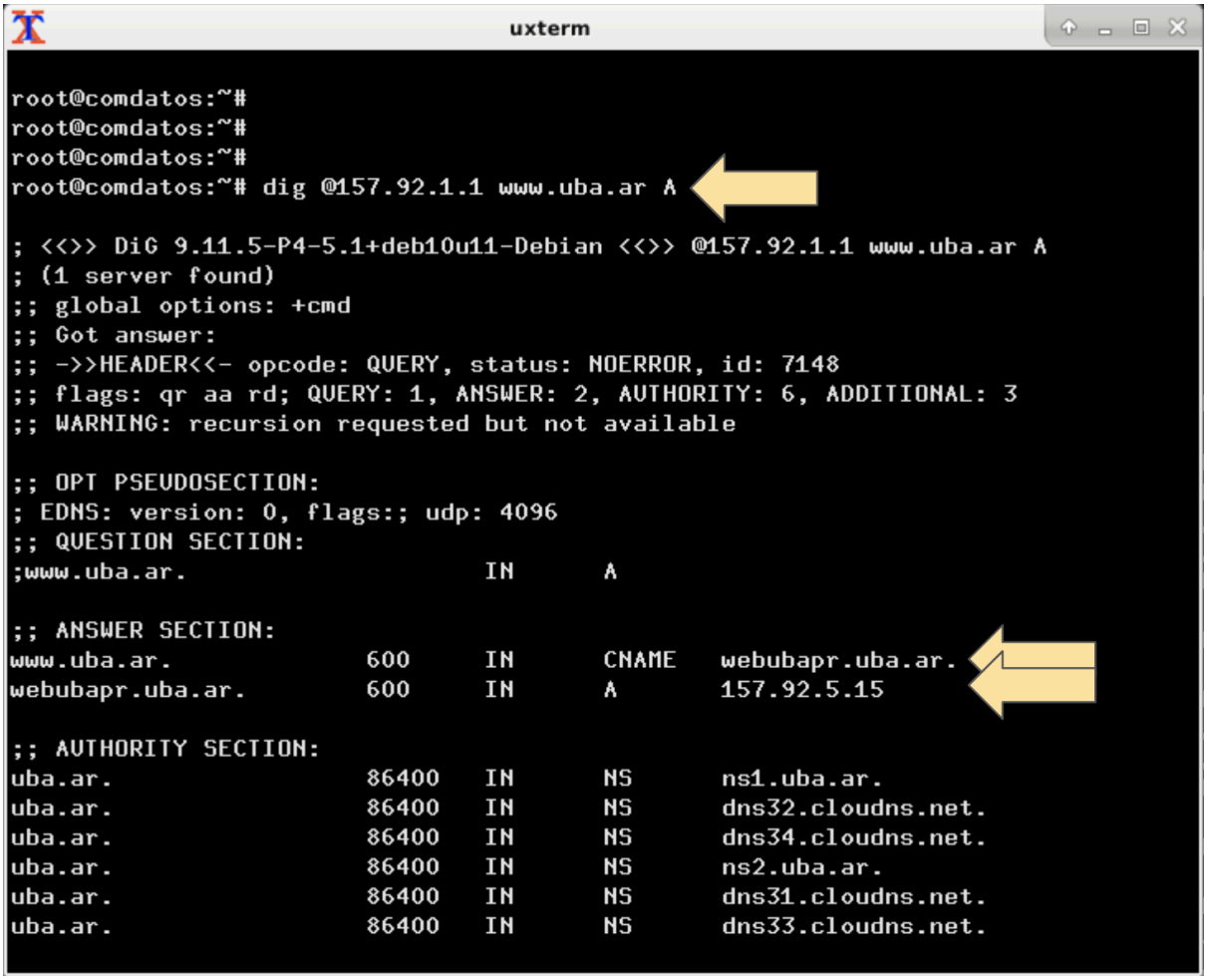



**Figura 5: consulta DNS y respuesta (pasos 6 y 7)**

**Paso 7.** En la respuesta que nos envía "**ns1.uba.ar"** podemos ver que "**www.uba.ar"** es un CNAME (canonical name) para "**webubapr.uba.ar"**, y éste último está asociado a la IP **157.92.5.15** mediante un registro de tipo "A".

**Paso 8.** Ahora que sabemos la dirección IP asociada a "**www.uba.ar"**, podemos enviar la respuesta al cliente de DNS que nos envió la consulta original.

Para validar este paso, vamos a cambiar de roles una vez más y asumir que somos el "**Requesting host**" del diagrama de la figura 1 de arriba. Es decir, de ahora en más ya no aplica lo que asumimos en el paso 1, ya que las consultas DNS generadas por dig estarán dirigidas a la IP del **Local DNS server** (que es la 10.0.2.3 como veremos abajo).

Lo cual nos lleva a la siguiente pregunta: en nuestro setup de VirtualBox, ¿quién nos provee de la función de "**Local DNS Server**" para que el sistema funcione? En esta versión de Debian, lo podemos averiguar revisando el archivo "**/etc/resolv.conf**":

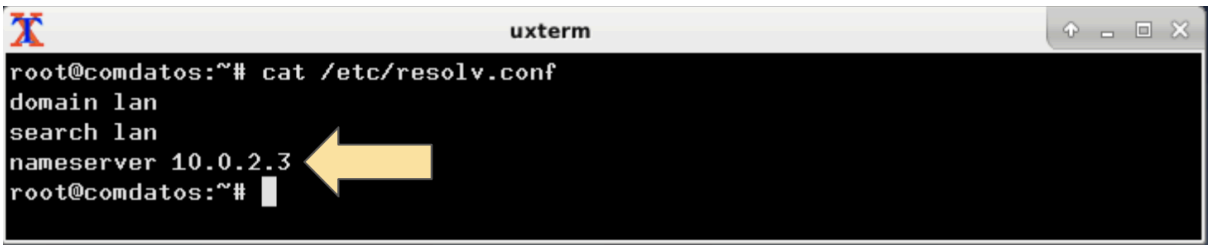



**Figura 6: contenido de `/etc/resolv.conf` en "Requesting host"**

Típicamente, cuando el sistema operativo inicializa la interfaz de red de la computadora obtiene su configuración de internet usando el protocolo DHCP. Es decir obtiene la IP del "**Local DNS Server**" (parámetro **`nameserver`** de **`/etc/resolv.conf`**) en la respuesta que el servidor DHCP le envía junto con el resto de la configuración de networking. Y con esta configuración el cliente de DHCP de Debian actualiza el contenido de **`/etc/resolv.conf`** al terminar la configuración de la interfaz.

Entonces, ahora que sabemos la IP repitamos una vez más la consulta y verifiquemos que la IP que **realmente** nos envía el verdadero **Local DNS Server** del entorno VirtualBox coincide con el análisis que hicimos en los pasos anteriores.

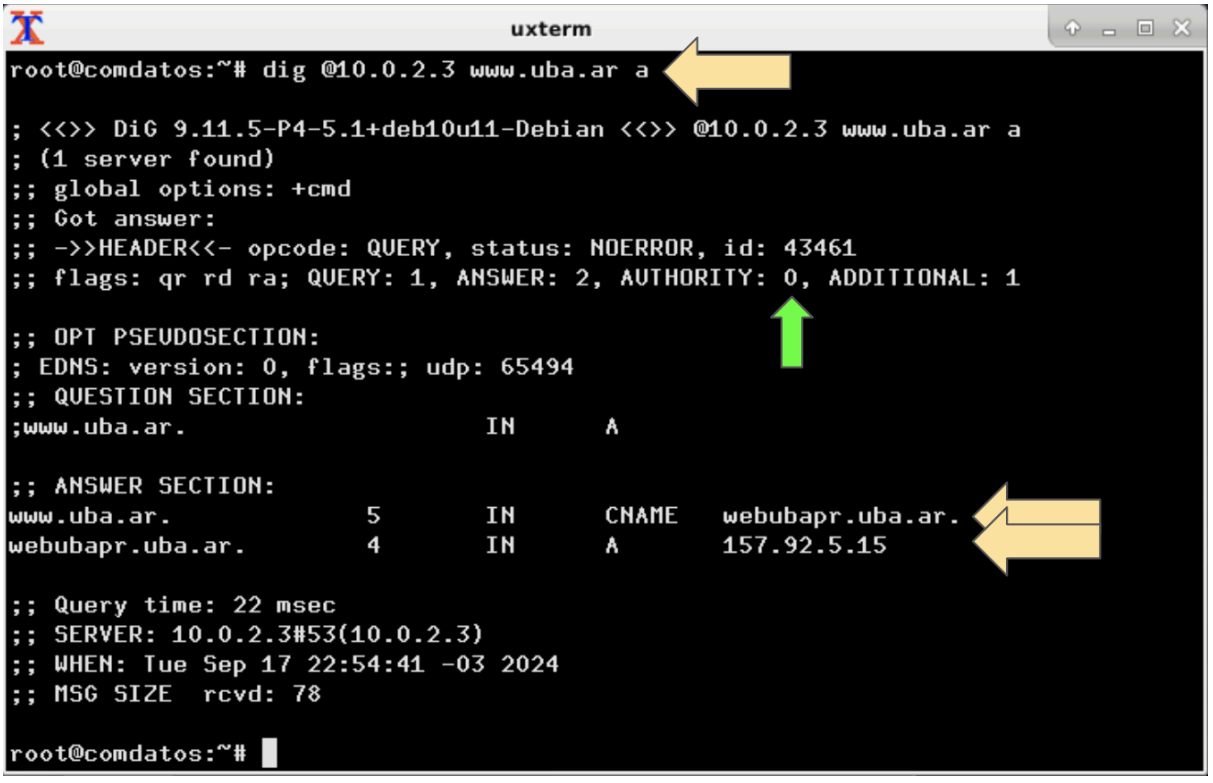



**Figura 7: consulta DNS y respuesta (paso 8)**

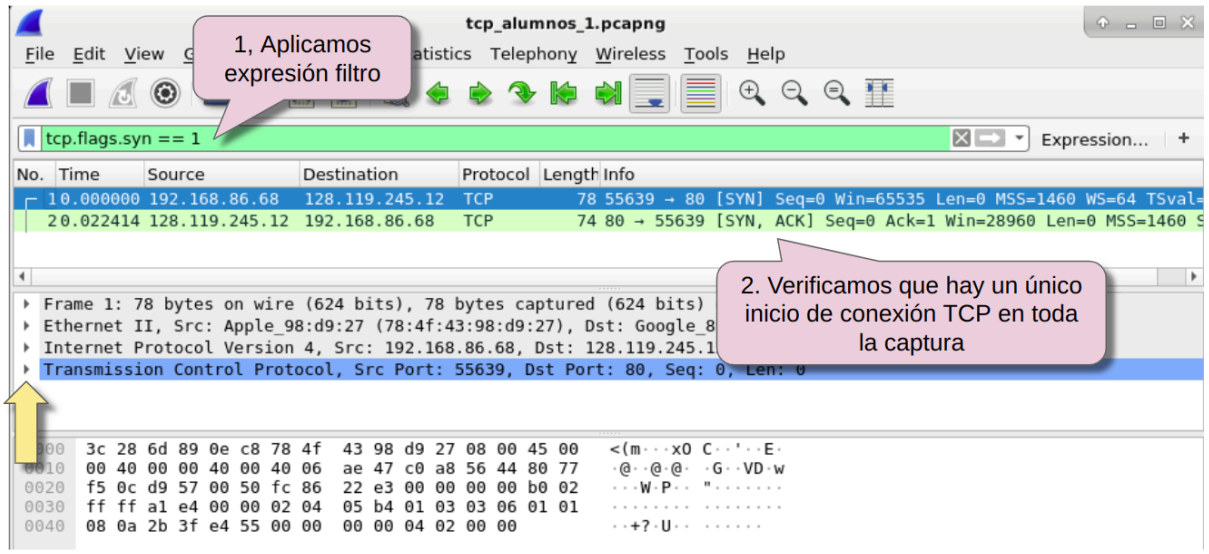
# Ejercicio de la capa de transporte

**1. Para la captura `tcp_alumnos_1_pcapng` del campus,**

**(a) ¿Cuál es la dirección IP y el port utilizado por el cliente?**
**(b) ¿Cuál es la dirección IP y el port utilizado por el servidor?**

Vamos a usar la VM del VirtualBox para resolver este ejercicio. Lo primero que vamos a hacer es abrir el Wireshark y cargar el archivo de la captura que nos pasaron junto con el enunciado del ejercicio.

Luego, vamos a verificar si realmente hay un único inicio de conexión TCP en toda la captura. Esto es importante porque, si hubiese múltiples inicios de conexión concurrentes en la misma captura tendríamos múltiples puertos y/o direcciones IPs involucradas.



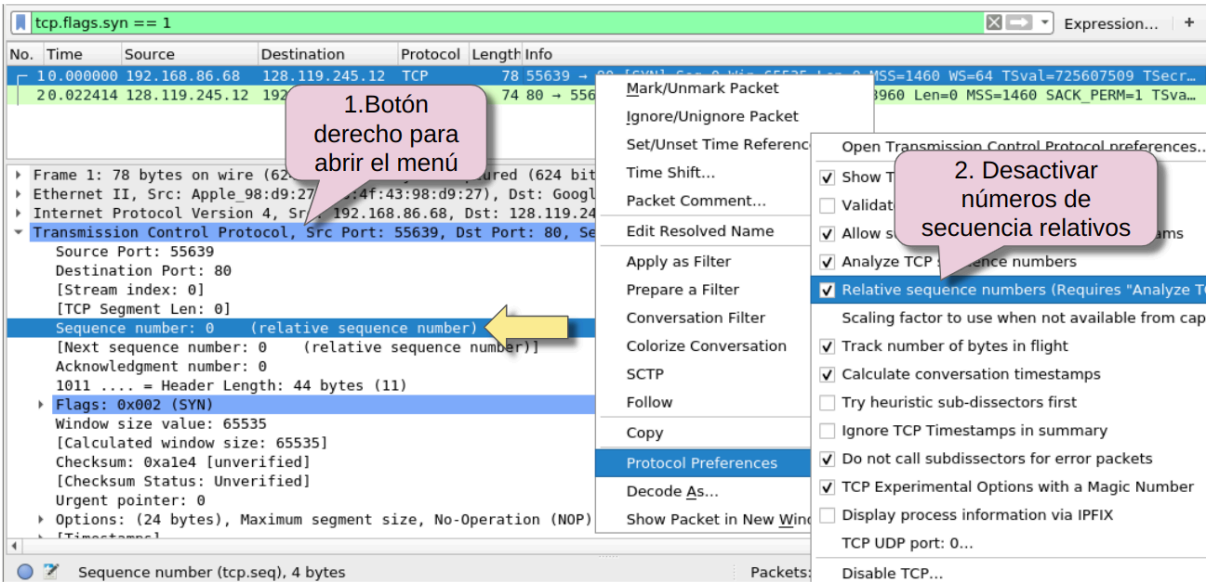

En la figura de arriba podemos ver que efectivamente la captura tiene un único inicio de conexión TCP. El puerto origen (lado cliente) es 55639, y el puerto destino (lado servidor) es 80 (que típicamente se corresponde con HTTP).

También vemos que la dirección IP de origen (lado cliente) es 192.168.86.68, mientras que la dirección IP (lado servidor) es 128.119.245.12.

*Antes de pasar a la siguiente pregunta, tocamos en el triangulito del encabezado de TCP para expandir esta parte del analizador de protocolos.*

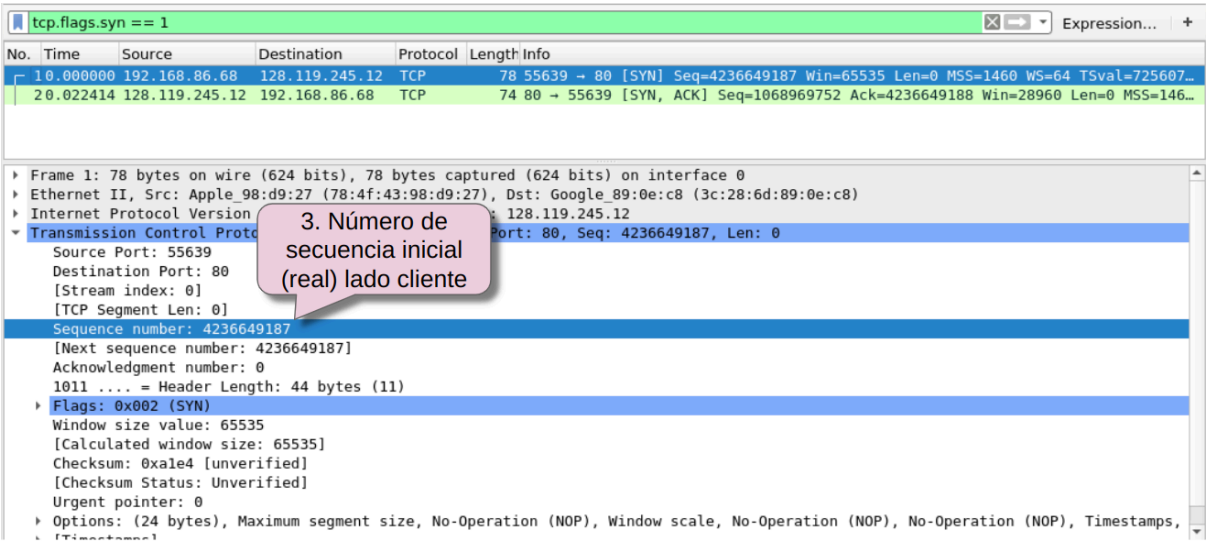
**(c) ¿Quién inició la sesión, el cliente o el servidor? ¿Con qué número de secuencia se inició?**

Como podemos ver en la captura, asumiendo que se trata de una conexión TCP usada para transportar una sesión HTTP (siendo rigurosos esto aún lo lo sabemos) la conexión TCP fue iniciada por el cliente HTTP ya que la misma fue apuntada al puerto 80 destino.

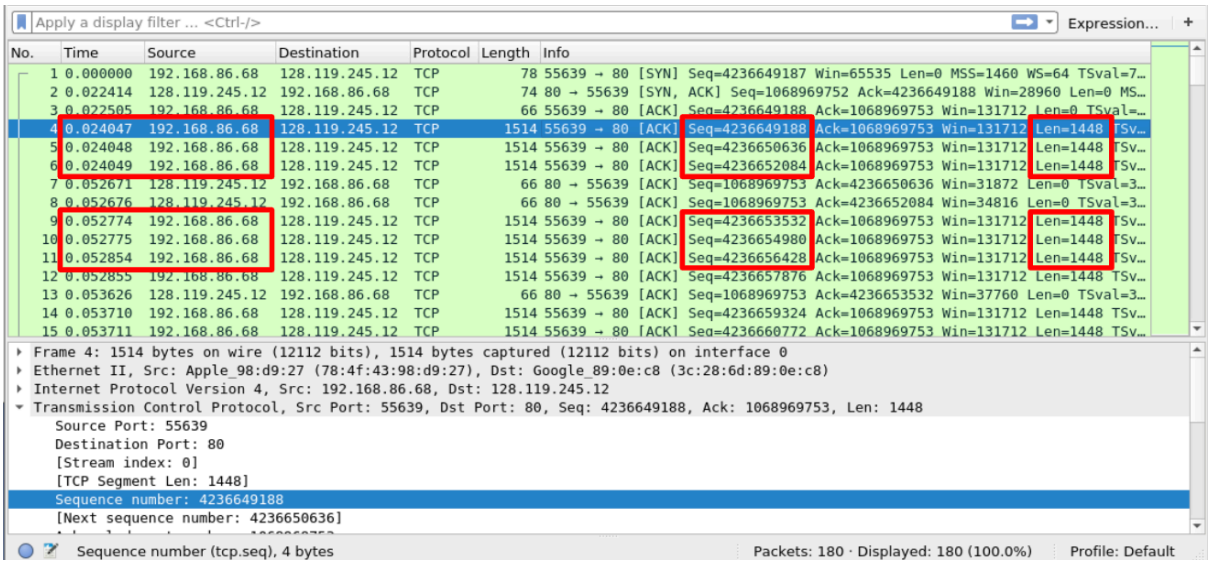



Luego de expandir el encabezado TCP, en la captura de arriba podemos ver que el número de secuencia usado por el cliente es 0 (cero). En realidad, lo que está sucediendo es que el analizador TCP de Wireshark implementa una función que se llama “números de secuencia relativos” para facilitar la visualización de los mismos (comienza a contar siempre desde 0 para simplificar la interpretación de los mismos).

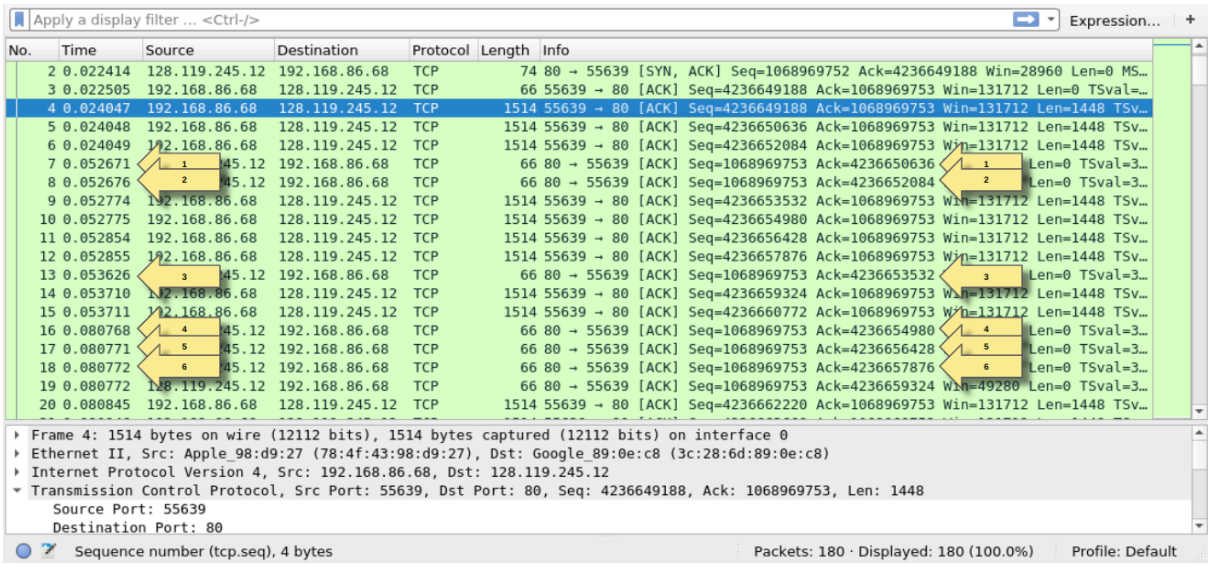
Entonces para visualizar el verdadero número de secuencia (indicado con una flecha amarilla en la captura de arriba), lo que hacemos es desactivar esta función como se muestra en la misma. Hecho esto la captura debería verse así:



**(d) ¿Cuáles son los números de secuencia de los primeros 6 segmentos de datos enviados? ¿En qué instantes de tiempo fueron enviados?**

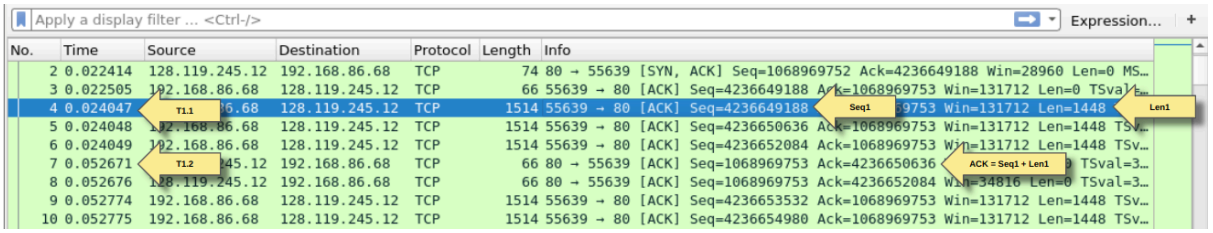



**¿Cuándo se recibió el segmento de ACK de cada uno de ellos?**




**¿Cuál es el RTT para cada uno de ellos?**

Interpretando al RTT como diferencia de timestamps, lado cliente, entre el envío de un segmento TCP y la recepción del ACK, vamos a calcular el mismo para el primer segmento de estos 6 que venimos analizando.




Entonces en la captura de arriba podemos ver que la diferencia T1.2 - T1.1 es de 28.624 milisegundos.

**(e) ¿Cuál es el valor del RTT estimado después de la recepción de cada ACK recibido? Considere que el primer RTT estimado es el RTT medido. Aplique la fórmula vista en clase considerando y = 0,125.**

TBD (resuelto completamente en la clase presencial 2024-09-19 usando el pizarrón).